ContractForge: The L0.5 Feedback Loop on gamma j

L0 → L0.5 loop → L1 → L2, and what the loop changes downstream

Bavya Sri Kilari (2025701029), Akshith K (2024111004), Vivek R (2019112020)
Autonomous Software Engineering, Monsoon 2026, IIIT Hyderabad

13 November 2026 Repository: github.com/Ohshit-ak/AUTOSE-Leaniers

1 Summary

Until now, an L0 spec that failed the L0.5 review was either dropped or regenerated once without being told why (`-on-l0-fail exclude|redo`). The new mode `-on-l0-fail loop` regenerates each failed spec with the L0.5 reasons and the rejected spec in the prompt, for up to two rounds. A new spec replaces the old one only if its L0.5 status improves. Whatever still fails is excluded before L1. We ran it on gamma j only, with L0 run once and then three L1+L2 repetitions.

- **The loop works at L0.5.** It fixed **6 of 9** failed specs (5 in round 1, 1 in round 2): 5 now pass and 1 is flagged. Failures drop from 9 to 3, and **5 more specs** enter L1 (57 → 62). It took 8.5 minutes, compared with 27 minutes for L0.
- **It adds about one verified spec.** Only 1 of the 5 repaired specs is verified end to end. Of the other four, two fail PO3 in L2 and two fail PO2 in L1, in all three repetitions (Issue 2).
- **The end-to-end count dropped from 52.7 to 47.3, but the loop did not cause it.** L0 was regenerated with a changed L0 prompt, and the new L0 was already weaker before the loop ran (57 pass vs 63). On the 17 specs whose text is the same in both runs, the results are identical (Issue 1).
- **Flagged specs never reach L1**, so turning a fail into a flag has no effect on the results (Issue 3).
- **The 3 remaining failures are compile errors**, and round 2 repeats round 1's feedback word for word (Issue 4).

1.1 Run setup

Document	`0000 - gamma j` (74 parsed entries)
Model	`qwen2.5-7b-instruct` (Q4_K_M, llama.cpp), temperature 0.2 (0.7 on restart); open-weight only
Harness	L0 v10.1 + L0.5 loop (2 rounds, `CF_L05_ROUNDS`); chain-v4 for L1+L2. Commit `8d03bd5`, clean tree
Job	Slurm 76346 (`runs/qwen-l05loop-gammaj.sbatch`), GTX 1080 Ti
Repetitions	L0 + loop: one run (seed 42). L1+L2: three repetitions (seeds 42, 43, 44), strict only (no fallback run)
Baseline	Frozen L0 `qwen-v10.1/rep1` (commit `b01f013`) and its chain run `qwen-chain-v4`
Outputs	`runs/0000 - gamma j/qwen-v10.1-l05loop/` (L0, `summary.md`) and `qwen-chain-v4-l05loop/` (L1+L2)

1.2 Terms used in this report

L0.5 status	*pass*: accepted, goes to L1. *flag*: suspicious, kept for review but *not* sent to the chain step. *fail*: rejected (does not compile, `post` accepts a forbidden output, a condition is always true, ...).
L0.5 loop	Regenerate every *fail* with the reasons and the rejected spec appended to the prompt. Keep the new spec only if its status is strictly better (fail < flag < pass). Repeat for at most 2 rounds, then exclude what still fails.
repaired spec	A spec that failed L0.5 and passes (or is flagged) after the loop.
end to end	Lean accepts one file with the spec, the architecture, PO1–PO3 and `theorem system_ok : Spec.Satisfies ...`.
weak spec	A spec that an implementation made only of copied input fields and constants satisfies.

2 What the loop did at L0.5

L0.5 status	frozen L0 (`b01f013`)	this run, before loop	this run, after loop
pass	63	57	**62**
flag	3	8	9
fail	8	9	**3**
total	74	74	74
into L1 (pass only)	61*	57	**62**

*The chain step takes specs that pass L0.5 *and* were checked. In the frozen run, 2 of the 63 passing specs (`3.4.3/S2`,

`UC-administrator-manage-plug-in-options`) were not checked, so 61 went to L1.

Entry	kind	rounds	after loop	downstream (3 reps)	L0.5 reason fed back
UC-sales-person-update-product-attributes	use case	1	pass	verified 3/3	four conditions $x \geq 0$ on `Nat` fields are always true
3.1.3/S2	requirement	1	pass	L2 PO3 fails	`post` accepts a forbidden output
5.3/R3	requirement	1	pass	L2 PO3 fails	`post` accepts every changed output
5.3/R4	requirement	1	pass	L1 PO2 fails	`post` accepts a forbidden output
5.5/R1	requirement	1	pass	L1 PO2 fails	does not compile (instance synthesis error)
UC-administrator-login-administrator	use case	2	flag	not sent to L1	`post` accepts a forbidden output
DD-shipping-date	data	2	fail	excluded	does not compile
UC-customer-add-item-to-cart	use case	2	fail	excluded	does not compile
UC-customer-checkout-an-order	use case	2	fail	excluded	does not compile

Table 1: The 9 entries that failed L0.5 before the loop. *rounds*: loop rounds spent on the entry (an entry fixed in round 1 is not retried).

The loop fixes the failures that a sentence can explain: an always-true condition, or a `post` that is too loose (4 of the 5 passes). It does not fix compile errors: 3 of the 4 compile failures are still failing after two rounds.

3 End-to-end results

Means over three L1+L2 repetitions (strict). The baseline uses the frozen L0; this run uses the new L0 after the loop.

	frozen L0 → `qwen-chain-v4`	this run → `qwen-chain-v4-l05loop`
specs into L1	61	62
L1 accepted	58.0 (58–58)	56.3 (56–57)
two-component architectures	1.0	0.3
L1 failures: po2 / identity / type	0.7 / 2.0 / 0.3	4.7 / 1.0 / 0
L2 failures: po3	5.3	9.3
end to end	**52.7** (52–53)	**47.3** (47–48)
on a non-trivial spec	30.7 (30–31)	26.0 (26–26)
requirement / data / use case	21.0 / 17 / 14.7	20.3 / 17 / 10.0
weak specs	24	23
minutes per repetition	11.5	14.0

Within this run, the loop's own contribution is +1: without it the 5 repaired specs would have been excluded, and only `UC-sales-person-update-product-attributes` is verified (in all three repetitions). So end to end would have been $47.3 - 1 = 46.3$ without the loop, and is 47.3 with it. Section 4 explains why the drop from 52.7 is not the loop's doing.

4 Issues, explained

Each issue gives what happens, the evidence from this run, why it matters, and a proposed fix. None of the fixes have been made yet.

Issue 1. This run cannot be compared with the frozen run

What happens. The job regenerated L0 instead of reusing the frozen `qwen-v10.1/rep1`. Both runs use seed 42, but the L0 prompt has changed since the frozen run: `prompt_hash` is `a4ace6f72b38` at `b01f013` and `08c8321aa2dd` at `8d03bd5`. So two things changed at once, L0 and the loop.

Evidence.

Specs in both chain runs (56)	specs	e2e, frozen	e2e, this run
identical Lean spec text	17	17.0	17.0
different Lean spec text	39	32.0	27.3

Where the spec text is the same, the results are the same. All of the drop comes from specs whose text changed: `1.4/S3`, `3.2.3/S1`, `5.3/R3`, `5.3/R4` and three administrator use cases (install patch, install plug-ins, remove plug-ins) are lost. `3.1.3/S3`, `SI-1.2` and `UC-administrator-add-user` are gained. Before the loop ran, the new L0 already had 6 fewer passing specs (57 vs 63).

Why it matters. A reader comparing the two headline numbers would conclude that the loop costs 5 verified specs. It adds one. The difference between the two L0 runs is larger than the effect being measured.

Proposed fix. Apply the loop to the *frozen* L0: copy `qwen-v10.1/rep1` and run only the check stage with `-on-l0-fail loop`. Then L0 is identical and the loop is the only change. Separately, rerun L0 with the current prompt over three seeds to see how much L0 varies on its own.

Issue 2. Passing L0.5 after feedback does not mean the spec is implementable

What happens. The loop makes a spec acceptable to the L0.5 review, but the repaired specs fail much more often later in the pipeline than the specs that passed on the first try.

Evidence. Repaired specs: 1 of 5 verified (20%). Specs that passed without the loop: 46.3 of 57 (81%). The four failures are the same in all three repetitions:

- `3.1.3/S2`, `5.3/R3`: routed single, L2 fails PO3 after 5 attempts.
- `5.3/R4`, `5.5/R1`: their new `post` relates two output fields, so they are routed to a split, and L1 fails PO2 in every repetition. The repaired `post` relates two output fields, which the router reads as an intermediate result.

In the frozen run, `5.3/R3` and `5.3/R4` were routed single and verified with L2 on the first attempt. Their repaired versions here fail. Those frozen specs were never rejected by L0.5, so the two runs do not show whether the feedback made the specs harder or the new L0 sample did. PO2 failures went from 0.7 to 4.7 per repetition, and these two specs account for 2 of them.

Why it matters. The loop raises the L0.5 pass count by 5, but that overstates its value: 4 of those 5 specs reach L1/L2 and still fail.

Proposed fix. Report the loop's effect at the end-to-end level, not at L0.5. Look at the regenerated `posts` of `5.3/R4` and `5.5/R1`: if the feedback ("`post` accepts a forbidden output") makes the model add an output-to-output relation, the split router will send the spec to L1 every time.

Issue 3. Flagged specs never reach L1

What happens. The chain step reads only specs with status *pass*. `UC-administrator-login-administrator` went from fail to flag in round 2, and the loop counted it as improved, but it is not used.

Evidence. 71 specs are pass or flag after the loop, but only 62 go to L1. All 9 flagged specs are left out, including the repaired one.

Why it matters. Counting fail → flag as an improvement makes the loop look more useful than it is in the results.

Proposed fix. Either loop on flags too (with the flag reason as feedback), or count only fail → pass as improved. Report the number of flagged specs next to the chain input.

Issue 4. Compile errors are not fixed, and round 2 repeats round 1

What happens. When a regenerated spec is not better, the loop keeps the *old* record. Round 2 then builds its feedback from that old record, so the model gets the same reasons and the same rejected spec as in round 1. It never sees why its round-1 attempt failed.

Evidence. `DD-shipping-date`, `UC-customer-add-item-to-cart` and `UC-customer-checkout-an-order` fail to compile in both rounds, each after 5 attempts per round. Round 2 fixed 1 of 4 entries, against 5 of 9 in round 1.

Why it matters. A second round with the same feedback is close to a blind retry. More rounds will cost time without fixing more.

Proposed fix. Build round-k feedback from the most recent attempt (its reasons and Lean error), while still keeping the better spec on disk. For compile errors, include the Lean error at its line, not only "does not compile".

5 Suggested order of work

1. **Measure the loop properly** (Issue 1): run the check stage with `-on-l0-fail loop` on a copy of the frozen L0, then chain-v4 ×3. This is a short job and gives the number to report.
2. **Feedback from the latest attempt** (Issue 4): a small change in `_l05_loop`, likely to help round 2.
3. **Decide what to do with flags** (Issue 3): loop on them or stop counting fail → flag as improved.
4. **Then run on the other 17 documents**: the loop is general (it uses only the L0.5 reasons), but report end to end with and without it, from the same L0.

Numbers from `runs/0000 - gamma j/qwen-v10.1-l05loop/summary.md`, `runs/0000 - gamma j/qwen-chain-v4-l05loop/rep{1,2,3}/l1` `runs/0000 - gamma j/qwen-chain-v4/summary.md` and `runs/qwen-l05loop-gammaj.log`.